ISSN 1009-3044
Computer Knowledge and Technology 电脑知识与技术
Vol.21, No.24, August 2025

E-mail：xsjl@dnzs.net.cn
http://www.dnzs.net.cn
Tel：+86-551-65690963 65690964

DOI:10.14004/j.cnki.ckt.2025.1228

前后端分离架构系统安全性设计与实现

栾征，张淑娴，倪婧祎

（上海杉达学院，上海 201209）

摘要：前后端分离架构因其技术栈选型的灵活性和前后端可并行开发的便利性成为现代Web应用开发的主流方案，但其无状态特性和API的完全暴露也对系统安全性提出了更高的要求。针对上述特点，研究从验证码机制、敏感信息加密传输与存储、用户令牌管理、权限控制、防暴力破解及防SQL注入等关键环节入手，提出了一套系统化的安全解决方案，实现了用户端、传输层、服务端、存储层的全链条纵深防御体系，为同类系统的安全实践提供了可复用的技术路径。

关键词：对称加密；非对称加密；权限控制；令牌；SQL注入；API限流

中图分类号：TP311　　**文献标识码：**A

文章编号：1009-3044(2025)24-0031-05

开放科学(资源服务)标识码(OSID)：

0 引言

近年来，随着互联网的快速发展，线上业务系统的数量迅速增长，随之产生的海量信息数据已成为数字经济时代的核心资产。信息安全直接关系到用户隐私、企业声誉乃至国家安全。2018年，欧盟颁布《通用数据保护条例(GDPR)》；2021年，中国颁布《个人信息保护法(PIPL)》。这些法规的出台，标志着国内外对信息安全的重视程度达到了新的高度。

作为数字经济的重要载体，Web应用(尤其随着前后端分离架构的广泛采用)因其开放性和高交互性，成为数据泄露与网络攻击的高发领域。表1中列出的近年来频发的数据泄露与越权访问事件，值得开发者警醒。

表1　前后端分离架构下部分信息安全事件及原因

时间	事件名称	原因
2019	Facebook用户数据泄露	根据电话号码查朋友接口缺乏限流机制、未启用验证码保护、未设计被查方确认功能
2020	微博用户数据泄露	用户查询接口未保证仅能获取当前用户信息、缺乏限流机制
2021	某电商平台JWT伪造漏洞	JWT加密密钥泄露(硬编码在前端)、普通用户伪造JWT后可拥有管理员权限
2022	某知名航空公司用户信息泄露	用户查询接口未保证仅能获取当前用户信息、缺乏限流机制、接口返回非必要信息
2022	Twitter用户数据泄露	根据邮箱查用户接口缺乏限流机制、未启用验证码保护、未设计被查方确认功能

1 前后端分离架构的安全性挑战

前后端分离架构以技术选型灵活、高性能与可扩展性、跨平台兼容性、研发效率高等优点，成为现代Web应用开发的主流方案。然而，相较于传统MVC架构的服务端渲染模式，该架构API接口的完全暴露特性更容易被工具扫描和获取完整的结构化数据(如JSON格式)，会话管理机制变革(如替代Session的Token体系)也更容易引发Token被伪造和密钥泄露问题。一旦上述漏洞被发现，若API的权限控制体系又相对脆弱、没有限流机制，往往会造成灾难性后果。表1中列举的信息安全事件均是上述原因综合被黑客利用的结果。

OWASP① Top 10(2021)[1]和OWASP API Security Top 10(2023)[2]报告显示，在Web应用及API安全威胁中，失效的访问控制、认证机制缺陷、暴力破解、注入攻击以及加密机制失效等风险持续位列高危漏洞前列。这些漏洞一旦被利用，可能导致数据泄露、权限提升甚至系统沦陷等严重后果。

2 前后端分离系统关键安全性设计与实现

为系统化应对这些安全挑战，需要构建覆盖身份认证与授权、数据传输安全、持久化数据保护的全生命周期防御体系。具体而言，系统应具有完备的授权体系与鉴权机制，接口应具备限流机制，数据应遵守最小化暴露原则，敏感信息应保证传输和存储加密并做好密钥管理。上述安全性相关设计应在系统设计初期前置纳入整体架构考量，尽量减少安全性问题的发生。以下将从架构设计原则(聚焦零信任架构设计、关键流程重点防御设计等)和工程实现(侧重加密算法选型、安全编码实践等技术落地)两个维度，详细

收稿日期：2025-04-30
作者简介：栾征(1982—)，男，山东济南人，助教，研究方向为应用软件设计与开发；张淑娴(2003—)，女，山西大同人，本科生；倪婧祎(2001—)，女，上海人，本科生。

中国知网 https://www.cnki.net

阐述针对性的解决方案。

2.1 验证码

验证码是一种应用于万维网的质询-响应测试，用于鉴别用户是人类还是计算机[3]，是系统安全性和公平性的重要保证。常见的验证码类型包括字符验证码、拼图验证码、短信验证码和邮箱验证码等。其中，短信验证及邮箱验证方式一般对接运营平台的短信网关或邮箱服务器，通过将验证码发送至用户手机或邮箱的方式完成验证，流程相对简单固定，安全性高，但依赖外部系统的响应。如果使用电子邮件验证，还需要用户登录邮箱查收验证码邮件，在用户体验上表现不佳。除此之外，上述两种验证方式在实际投产应用中也会产生部分费用。因此，保留一种不依赖外部系统且安全性较高的验证方式是必要的。对比几种常见的验证码类型(见表2)，传统字符验证码因其容易被OCR识别但不易被人识别而逐渐被淘汰；拼图验证码虽可以被计算机视觉技术识别，但通过对拼图行为分析可以有效识别出这种人机差异(计算机进行拼图时的运动轨迹通常是机械的，例如匀速、直线等)。因此，设计中采用拼图验证码+行为分析的方式作为验证码的首选方案。

表2　常见验证码的类型、安全性、用户体验和成本

类型	安全性	用户体验	成本
传统字符验证码	低	差	低
拼图验证码	中	好	低
拼图验证码+行为分析	较高	好	低
短信验证	高	中	高
邮箱验证	高	差	高

拼图验证码+行为分析需要前后端协作完成验证过程，具体过程如图1所示。当用户请求登录时，后端随机选取图库中的一张图片，对图片进行随机切割，将切割出的拼图块与切掉拼图块后的原图返回给前端，由前端将拼图呈现给用户(见图2)。在用户进行拼图的过程中，前端不断收集用户行为数据。用户完成拼图后，前端将用户的拼图行为数据及拼图位移信息等传给后端。后端对用户行为及用户最终拼图结果进行分析判定，并将判定结果返回前端。如果验证无误，前端将用户之前输入的账号、密码、拼图位移信息提交给后端进行登录验证。若没有拼图位移信息或者拼图位移信息错误，后端验证接口将拒绝此次用户密码的校验。特别需要注意的是，后端在登录过程中对验证码完成二次验证后，须将验证码销毁，以免重复使用。

2.2 敏感信息加密传输

在用户登录过程中，通常需要提交账号、密码等敏感信息。服务端可通过强制HTTPS访问、关闭弃用的传输层安全协议(例如SSL1.0、SSL2.0、SSL3.0、TLS1.0、TLS1.1等)，仅保留安全的加密套件(例如TLS_AES_256_GCM_SHA384、TLS_CHACHA20_POLY1305_SHA256、TLS_AES_128_GCM_SHA256等)等方式，尽可能保护数据以安全的加密方式传输。

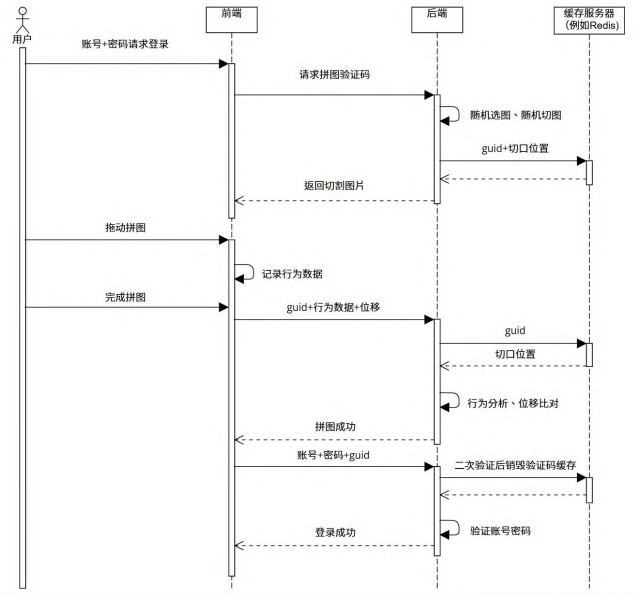

图1　账号密码登录时序图

图2　拼图验证码UI实现示例

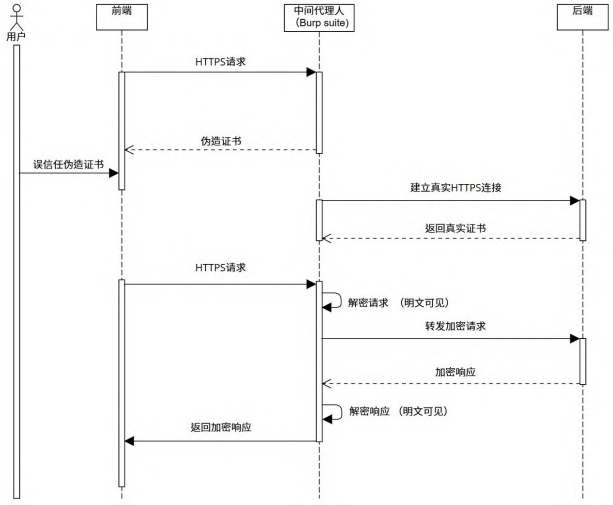

图3　中间人代理攻击时序图

虽然上述防御措施能够有效降低传输过程中的信息泄露，但中间人攻击(MITM)仍可能构成威胁[4]。如图3所示，攻击者通过在用户浏览器与服务器之间建立代理连接，诱导用户安装伪造的根证书。一旦得逞，攻击者既能获取服务器的真实证书，又可解密HTTPS传输的加密数据，从而窃取用户的登录凭证。

中国知网 https://www.cnki.net

用户信任伪造的证书是导致敏感信息被窃取的根本原因，因此，用户安全意识教育也是安全防御体系中的重要一环。技术层面上，客户端应在应用层对敏感信息进行加密后传输，使中间人即使能够解密HTTPS的密文，也仍然无法直接获取敏感信息，从而起到保护作用。但如果使用对称加密算法，加密密钥会在前端暴露，容易被黑客获取，失去加密的意义。因此，该场景下需要使用非对称加密算法(如RSA) 进行加密。RSA加密算法的原理是公钥加密，私钥解密。公钥可以公开发给客户端，私钥必须保留在服务端，不可泄露。客户端利用公钥对敏感信息(例如密码) 加密后发送，服务端使用私钥解密后进行验证。具体设计如图4所示。

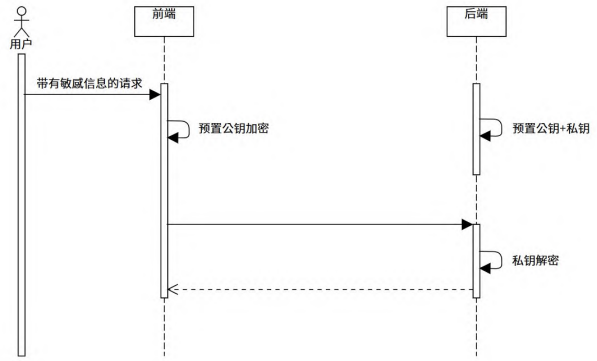

图4　RSA加密解密时序图

2.3 敏感信息加密存储

用户密码及部分用户个人信息例如手机号、身份证号、银行卡号等信息，持久化存储在数据库中，一旦发生泄露，将造成严重的信息安全事件。为了增强上述信息的存储安全性，对敏感信息应加密后再存储。由于该场景的加密与存储行为均发生在服务端，因此应当选用高安全性的对称加密算法进行加密。其中，AES128加密算法以其密码学安全性和抗暴力破解特性被广泛应用。该加密算法的常见工作模式、原理、优缺点见表3。具体实现中推荐选用GCM模式，须避免使用ECB模式。

表3　AES128常见工作模式对比

模式	原理简述	优点	缺点
ECB	明文分组独立加密	简单快速，并行处理，无初始化向量(IV) 需求	相同明文加密后相同密文，安全性低
CBC	每个明文块与前一个密文块异或后加密，第一个块与IV异或	隐藏明文，相同明文加密后不同密文，比ECB安全	无法并行处理，需要填充，需要IV，一旦出错，错误会传播至两个块
CTR	使用计数器作为加密器输入，计数器输出与明文异或生成密文	可并行处理，无须填充，错误不传播	需要保证计数器唯一性
GCM	结合CTR模式和伽罗瓦域认证，提供加密和认证	提供数据加密和完整性验证，可并行处理，效率高	需要IV，需要保证IV的唯一性

对称加密的密钥管理是关键。在成本可控的前提下，优先选择云托管方式管理密钥。AWS、Azure、GCP等主流云服务商均提供专业的密钥管理解决方案。密钥分为主密钥(KEK) 和数据加密密钥(DEK) ，其中KEK仅用来对DEK进行加密，DEK用来对数据进行加密解密。KEK不离开云服务器，使用KEK加密后的DEK存放在本地数据库中(见图5) 。如无法接入云资源，KEK需要单独存放在特定的服务器，并设置文件的最小访问权限，确保仅有特定用户(例如应用程序) 可访问。密钥需要设置版本号，以支持流转机制(例如每6个月更换) 。

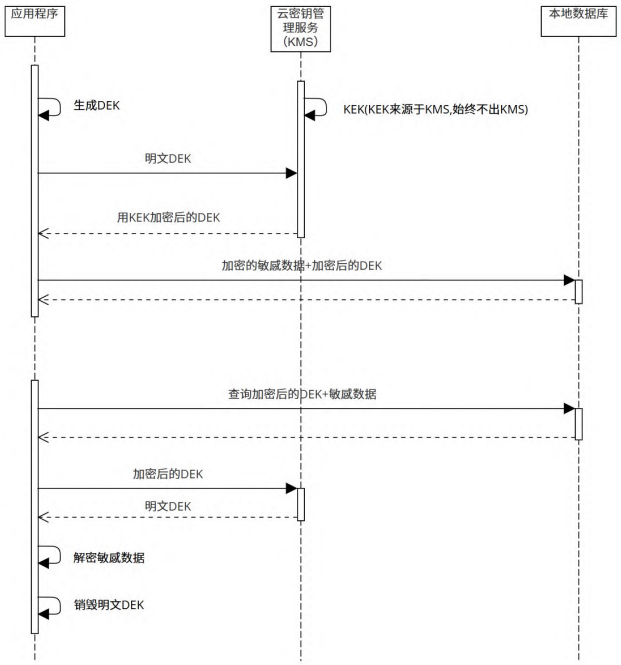

图5　使用云服务管理KEK时序图

2.4 用户令牌(Token)

用户密码是用户身份的关键敏感信息，应尽可能减少暴露。除登录接口需要携带用户密码信息外，其余接口利用Token(例如JWT) 保证请求的合法性。Token在用户首次成功登录系统后，由服务端发放给客户端，客户端应缓存Token,以便后续请求服务端服务时，携带在请求中。Token包括访问令牌(AccessToken) 和刷新令牌(RefreshToken) 。AccessToken是短期的有效令牌，过期时间通常在15～30分钟，用来访问服务端受保护的资源；RefreshToken是长期的有效令牌，过期时间通常在几天至几十天，用于获取新的AccessToken。用户通过账号、密码方式登录或者由于AccessToken过期触发刷新Token，系统将颁发新的AccessToken和RefreshToken(见图6) 。这种设计既保证了安全性，又避免了需要频繁登录导致用户体验不佳。

中国知网 https://www.cnki.net

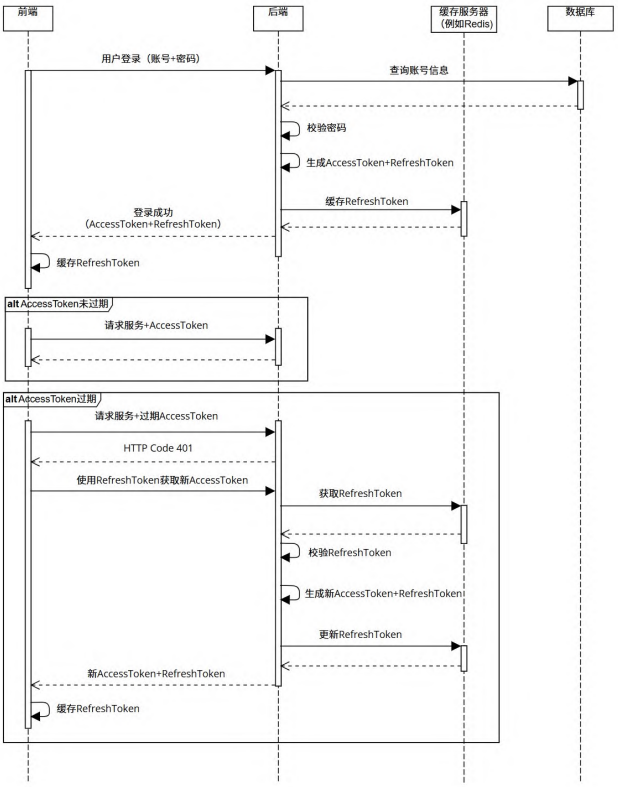

图6　用户Token交互时序图

2.5 权限分离的访问控制

Web应用系统一般采用RBAC(Role-Based Access Control,基于角色的访问控制)模型将用户角色与访问权限相关联。根据用户角色决定用户在系统中的访问权限,以便实施授权和安全约束。该模型通过增加角色这一概念,将权限与角色之间、角色与用户之间建立关联,提高了授权的灵活性,更容易实现权限的最小化原则,以及职责分离等各种安全策略[5]。

系统权限设计与实现主要是基于RBAC模型,数据库设计如图7所示。但传统的基于RBAC模型的访问控制需要超级用户为其他用户授权,导致超级用户权限过大,且易出现过度授权。针对这一问题,系统设计提出在初始化时内置默认角色,对系统功能先行划分控制。例如,表4给出了一种默认角色划分的举例。用户管理员、审计管理员、操作管理员等3个默认角色分别从用户管理、审计、系统操作层面切分系统功能,默认角色分配管理员为第4个默认角色,当且仅当用户具有该角色时,可以为其他用户分配默认角色。系统通过逻辑控制默认角色互斥,同一个用户最多只能分配1个默认角色。拥有默认角色的用户可以在默认角色的权限范围内创建子角色并为子角色授权,但授权功能不扩散,即子角色只能拥有默认角色中除授权功能外的其他功能。一个用户可以关联多个角色,但多个角色必须归属于同一个默认角色,以保证权限不可跨默认角色流动。

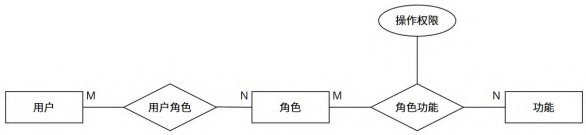

图7　用户权限E-R图

表4　一种内置默认角色划分示例

默认角色	功能
默认角色分配管理员	负责为用户分配默认角色
用户管理员	负责创建用户、修改用户信息,启用/禁用用户,在用户管理员权限范围内创建和授权子角色。授权功能本身不可授权
审计管理员	负责审计系统日志,在审计管理员权限范围内创建和授予子角色。授权功能本身不可授权
操作管理员	负责系统业务操作,在操作管理员权限范围内创建和授予子角色。授权功能本身不可授权

上述设计可以保证拥有特定角色的用户只能在页面上看到和操作该角色拥有权限的功能。但攻击者一般会绕过前端,通过直接调用后端接口的方式进行攻击。如API接口未有效根据用户身份进行鉴权,则会出现垂直越权(例如普通用户可以调用管理员接口)与水平越权(例如A用户查询修改B用户的数据)两类安全性问题。因此,后端接口需要在每次被调用时,首先根据用户id定位该用户是否具有访问该接口的权限,确保接口不接受越权(垂直越权)请求,具体流程如图8所示。另外,对于普通用户,系统也需要确保用户id与用户所操作数据所有者的id一致,以杜绝水平越权问题。

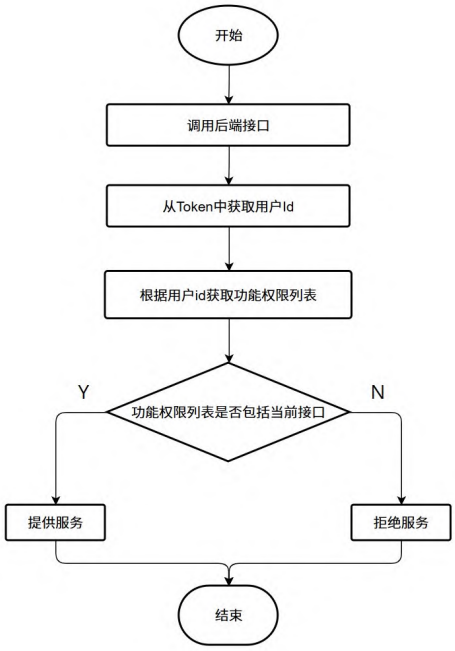

图8　接口权限校验流程图

2.6 防暴力破解

用户注册与登录接口是防暴力破解攻击的重点,

中国知网 https://www.cnki.net

须综合使用强制复杂用户密码(例如密码长度8位以上,必须包括大写字母、小写字母、特殊字符和数字)、多次密码错误锁定账号一段时间、验证码等策略共同建立防护机制。对于其他接口,可采用基于令牌桶算法的限流机制,并在API被调用时根据接口是否要求用户登录分别采用基于用户的限流或基于IP地址的限流,以确保尽量精确地控制被限流对象,具体流程如图9所示。

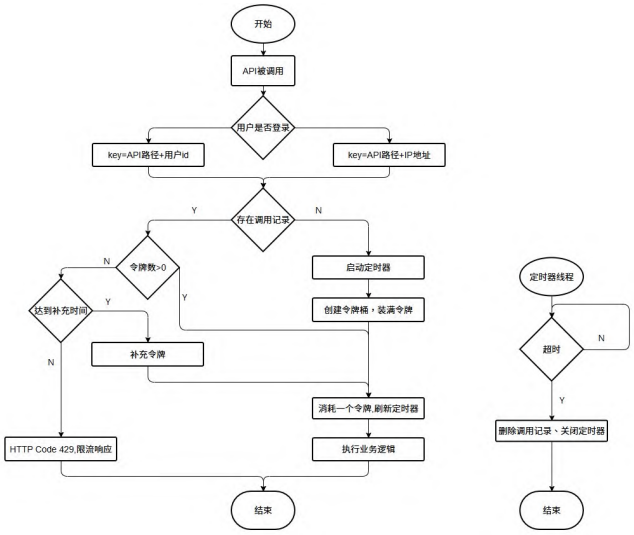

图9　WebAPI限流流程图

2.7 防SQL注入

防SQL注入主要依靠ORM框架保证。目前主流的ORM框架例如Java的MyBatis、.NET的Entity Framework、Python的Django等均通过预编译语句、参数化查询参数等方式实现了防SQL注入。开发者依赖成熟ORM框架提供的安全接口进行开发即可有效避免SQL注入。

3 结束语

本研究聚焦前后端分离架构下的系统安全设计与实现,从验证码机制、敏感信息加密传输与存储、用户令牌管理、权限控制、防暴力破解及防SQL注入等关键环节入手,提出了一套系统化的安全解决方案。通过引入验证码和限流策略,有效抵御自动化脚本攻击,提升了系统的抗暴力破解能力。采用HTTPS协议与非对称加密技术保障了数据传输的机密性,结合AES等算法实现敏感数据的加密存储,降低了数据泄露风险。基于JWT的令牌机制与基于角色的访问控制模型,实现了用户身份的动态校验与细粒度访问控制。特别地,对传统RBAC模型下存在超级用户权力过大和易出现过度授权的问题,给出了针对性的设计与解决方案。此外,通过参数化查询与ORM框架的运用,从根源上阻断了SQL注入漏洞。这些设计方案在保障系统功能性的同时,保证了系统的安全性,为同类系统的安全实践提供了可复用的技术路径。然而,随着技术演进和环境变化,前后端分离架构的安全防护需持续演进。未来须进一步研究AI对Web API带来的新挑战,例如智能自动化攻击、防御机制绕过、深度身份伪造等。在现有安全防护基础上,结合AI手段综合防御,逐步从传统的规则防御转向“AI对抗AI”的防御新范式。

注释:

① OWASP(Open Web Application Security Project,开放网络应用安全项目)是一个非营利性组织,专注于提高软件和网络应用的安全性.

参考文献:

[1] OWASP.OWASP Top 10[EB/OL].[2023-10-20].https://owasp.org/Top10.
[2] OWASP.OWASP API Security Top 10[EB/OL].[2023-10-20].https://owasp.org/API-Security/editions/2023/en/0x11-t10.
[3] VON AHN L,MAURER B,MCMILLEN C,et al.reCAPTCHA:human-based character recognition *via* Web security measures[J].Science,2008,321(5895):1465-1468.
[4] 邓真,刘晓洁.HTTPS协议中间人攻击的防御方法[J].计算机工程与设计,2019,40(4):901-905.
[5] 雷惊鹏.一种基于角色访问控制模型的设计与实现[J].长沙大学学报,2022,36(5):15-23.

【通联编辑:谢媛媛】

中国知网 https://www.cnki.net